

SP-201 Red Multi-Agent Research & Fact-Checking Pipeline

PROJECT DEVELOPMENT DOCUMENT

CS4850 | Section 2 | Spring 2026

March 1, 2026

Professor Perry

 Melike Ozcelik Team Leader Developer	 Raaid Khan Developer Documentation	 Nha Phan Developer	 Afra Kurudirek Documentation
---	---	--	---

1. Implementation Detail Narrative.....	2
1.0 System Integration & Workflow Overview.....	3
Operational Workflow	3
Global Design Decisions	3
1.1 Manager Agent.....	4
Conceptual Overview.....	4

Development Process	4
The Manager Agent was implemented as a Python module using Google ADK. Its development included:.....	4
1.2 Search Agent	4
Conceptual Overview	5
Development Process	5
1.3 Synthesizer Agent	5
Conceptual Overview	5
Development Process	5
1.4 Fact-Checker Agent	6
Conceptual Overview	6
Development Process	6
1.5 Memory Service.....	6
Conceptual Overview	6
Development Process	6
2. Database and External Connections	7
2.1 Connection Overview:	7
2.2 Configuration Details:.....	7
2.3 Integration Logic:.....	8
3. Project Setup and Environment Configuration.....	8
3.1 Prerequisites.....	8
3.2 Installation Steps	9
3.3 Verification	9
4. Development Conclusion or Summary	9

1. Implementation Detail Narrative

The SP-201 Red Multi-Agent Research & Fact-Checking Pipeline is implemented as a hierarchical Manager–Worker multi-agent system, consistent with the architectural decisions documented in the SRS and SDD.

The system integrates five primary components: the User Interface (CLI or web-based), Manager Agent, Search Agent, Synthesizer Agent, Fact-Checker Agent, and the Session State Manager.

Communication between components follows a structured, sequential message-passing model. The User Interface collects a natural-language research prompt and passes it to the Manager Agent. The Manager Agent decomposes the prompt into structured sub-questions and delegates them to the Search Agent. The Search Agent retrieves structured data from external APIs (e.g., DuckDuckGo) and returns citation-preserving results to the Manager Agent. These results are forwarded to the Synthesizer Agent, which generates a logically organized draft while preserving citation mappings. The draft is then sent to the Fact-Checker Agent for validation. The Fact-Checker Agent compares generated claims against retrieved source material and returns a validation status. If inconsistencies are detected, the Manager Agent initiates a controlled rollback to either the Search or Synthesizer stage. We treat all sub agents as tools rather than individual specialists, so the orchestration goes from agent to agent to ensure the workflow pipeline is not interrupted.

All communication occurs through structured data objects (JSON-like schemas defined during architecture design in Phase 2 Project Plan. Sequential execution is enforced to prevent race conditions and maintain deterministic state transitions.

The system integrates the following external technologies:

- Python 3.10+
- Google ADK for agent orchestration
- Gemini API (Free Tier models such as gemini-1.5-flash)
- Public search APIs for external data retrieval

The integration ensures that each agent operates independently while remaining coordinated through the Manager Agent’s orchestration logic.

1.0 System Integration & Workflow Overview

Operational Workflow

- A single system execution (“run”) proceeds as follows:
- The user submits a complex research question through the CLI interface.
- The Manager Agent analyzes the prompt and decomposes it into at least three structured sub-questions.
- Each sub-question is submitted to the Search Agent.
- The Search Agent calls an external search API, parses the returned JSON data, extracts relevant text and citation metadata, and returns structured results.
- The Manager Agent aggregates search results and forwards them to the Synthesizer Agent.
- The Synthesizer Agent constructs a structured prompt and uses the Gemini API to generate a cohesive markdown-formatted draft.
- The draft and original source data are passed to the Fact-Checker Agent.
- The Fact-Checker Agent compares generated claims against retrieved source material.
- If unsupported statements are found, the Manager Agent reinitiates the Search or Synthesis phase.
- If validation succeeds, the final research report with clickable citations is displayed to the user.
- The workflow follows a deterministic state sequence:
- Pending → Decomposing → Searching → Synthesizing → Fact-Checking → Finalizing

Global Design Decisions

Several global technical decisions shaped the development:

- **Manager–Worker Architecture:** A centralized orchestration model was selected to improve modularity, traceability, and verification control.
- **Sequential Execution Model:** Agents execute in controlled sequential phases rather than parallel execution to prevent race conditions and simplify debugging.
- **Structured JSON Data Objects:** All inter-agent communication uses structured schemas to ensure consistency and citation traceability.

- **Global Error Handling Strategy:** All API calls include structured exception handling and retry logic for transient failures and rate limits.
- **Token Management Controls:** Because the system uses free-tier Gemini models, prompt size limits and truncation safeguards were implemented to prevent token overflow.
- **Environment Variable Security:** API keys are stored in environment variables rather than hard-coded in the source code.

1.1 Manager Agent

Conceptual Overview

The Manager Agent is the core orchestration component of the system. It controls workflow progression, manages state transitions, decomposes prompts into structured sub-questions, delegates tasks to worker agents, and ensures validated output is returned to the user.

Development Process

The Manager Agent was implemented as a Python module using Google ADK. Its development included:

- Designing prompt decomposition logic using Gemini.
- Defining structured schemas for sub-question generation.
- Implementing a deterministic state machine to track execution phases.
- Creating rollback logic to handle failed validation loops.
- Integrating calls to Search, Synthesizer, and Fact-Checker agents in a controlled sequence.

State transitions are enforced programmatically to prevent invalid workflow movement.

1.2 Search Agent

Conceptual Overview

The Search Agent is responsible for retrieving relevant external information based on structured sub-questions. It ensures that retrieved data includes source titles and URLs for citation tracking.

Development Process

The Search Agent was implemented using a FunctionTool integration within Google ADK. Development included:

- Creating an API wrapper for public search endpoints (e.g., DuckDuckGo).
- Parsing JSON responses from the search API.
- Extracting content, source titles, and URLs.
- Structuring results into citation-preserving objects.
- Implementing retry logic for rate limits and temporary API failures.

1.3 Synthesizer Agent

Conceptual Overview

The Synthesizer Agent integrates structured search results into a coherent research draft while preserving citation references. Output is formatted in markdown for readability.

Development Process

The Synthesizer Agent was implemented using Gemini via Google ADK. Development steps included:

- Constructing structured prompts using aggregated search results.
- Controlling token usage to remain within free-tier constraints.
- Formatting output as markdown with inline citation markers.
- Ensuring no unsupported claims are generated beyond retrieved data.

1.4 Fact-Checker Agent

Conceptual Overview

The Fact-Checker Agent validates the synthesized draft by comparing claims against retrieved source data. Its purpose is to reduce hallucinated outputs and improve research reliability.

Development Process

The Fact-Checker Agent was developed using Gemini-based evaluation prompts. Implementation included:

- Extracting factual claims from the synthesized draft.
- Comparing claims against original retrieved content.
- Flagging unsupported statements.
- Returning a validation status to the Manager Agent.
- Triggering controlled re-execution when necessary.

1.5 Memory Service

Conceptual Overview

The Memory Service enables the system to retain and recall information from past conversations. Without memory, each session starts with no prior context. By integrating memory, the Manager Agent can reference previous research queries and results, improving continuity and reducing redundant searches. Memory is implemented using Google ADK's built-in InMemoryMemoryService for prototyping and local development.

Development Process

The Memory Service was integrated into the Manager Agent using Google ADK's memory tools. Implementation included:

- Importing the load_memory tool from Google ADK to allow the Manager Agent to search past conversations.
- Implementing an after_agent_callback function that automatically saves each completed session to the memory store.

- Adding an instruction to the Manager Agent to use the load_memory tool when past conversation context may be relevant.
- Configuring the system to use InMemoryMemoryService as the default memory backend, which requires no additional setup.

Memory is stored in application memory during the server process lifetime. Each completed conversation is automatically ingested into the memory store. When a new query is submitted, the Manager Agent can search the memory store using keyword matching to retrieve relevant context from prior sessions.

Known limitations of the current memory implementation include:

- Memory is not persistent across server restarts (InMemoryMemoryService stores data in RAM only).
- Search capability is limited to basic keyword matching.
- For production use, migration to VertexAiMemoryBankService is recommended for persistent storage and semantic search.

2. Database and External Connections

2.1 Connection Overview:

The system primarily connects to external data sources rather than a traditional relational database. The primary external connections include:

- Gemini API (Google AI Studio Free Tier)
- DuckDuckGo (or similar public search API)

A MySQL database may optionally be used for storing session logs or search results, but database integration is not required for core functionality.

2.2 Configuration Details:

Environment variables required:

- GEMINI_API_KEY – Google AI Studio API key
- SEARCH_API_ENDPOINT – Search API base URL
- SEARCH_API_KEY (if required by provider)

Example configuration:

```
export GEMINI_API_KEY=your_api_key_here
```

```
export SEARCH_API_ENDPOINT=https://api.duckduckgo.com/
```

All API communication occurs over HTTPS.

2.3 Integration Logic:

The system communicates with external APIs using REST-based HTTPS requests.

- The Search Agent submits GET requests to retrieve JSON-formatted search results.
- The Synthesizer and Fact-Checker Agents submit structured prompts to the Gemini API via POST requests.
- Responses are parsed, validated, and transformed into structured internal objects before being passed to the next agent.

All external communication is wrapped in structured error-handling blocks.

3. Project Setup and Environment Configuration

3.1 Prerequisites

List necessary software, versions, and dependencies (e.g., Python 3.10+, required hardware specs).

1. Software / versions
 - a. VS / Python 3.10
 - b. Git
 - c. Google ADK
 - d. Microsoft Access 2016
 - e. PIP
2. Dependencies
 - a.
3. Hardware
 - a. Google API

3.2 Installation Steps

1. Clone the repository.

Git clone https://github.com/PJ201-Senior-Project-2026/agent_server.git

2. Install dependencies.

pip install -r requirements.txt

3. Configure environment variables.

Go to agents, create .env file that looks like .env.example and add required environment variables

GOOGLE_API_KEY = ...

3.3 Verification

The system is correctly set up if:

- The CLI prompts the user for a research query.
- Workflow stages are printed in sequence (Decomposing → Searching → Synthesizing → Fact-Checking → Finalizing).
- A formatted markdown research report with citations is displayed.

If API configuration is incorrect, the system will return a structured error message.

4. Development Conclusion or Summary

- The current build successfully implements hierarchical multi-agent coordination, external data retrieval, structured synthesis, and automated fact-checking. The system operates within free-tier API constraints and supports a full end-to-end research workflow.

Known limitations include:

- Dependency on external API availability.
- Free tier rate limits and token constraints.
- Sequential execution may increase latency for complex prompts.
- Validation accuracy depends on the quality of retrieved sources and LLM evaluation.

Future improvements may include parallel agent execution, result caching, improved UI visualization, and expansion to additional validation agents (e.g., bias detection or citation formatting).